

Experiment 3: Regression Analysis using Linear and Regularized Models

NATHANIEL CHRISTIAN
Reg. No. 3122247001037

August 1, 2026

1 Aim and Objective

The aim of this experiment is to implement linear and regularized regression models – Linear Regression, Ridge, Lasso, and Elastic Net – to predict the sanctioned loan amount from applicant and loan-related features, and to study the effect of regularization strength on model performance and generalization. Specifically, the experiment evaluates:

- Baseline (unregularized) Linear Regression versus the three regularized variants.
- The effect of regularization strength (α) and, for Elastic Net, the L1/L2 mixing ratio.
- Generalization behaviour via 5-fold cross-validation.
- The effect of regularization on coefficient magnitudes.

2 Dataset Description

The dataset used is the Kaggle *Predict Loan Amount Data* set, containing loan application records with a mix of numerical and categorical features. The target variable is the sanctioned loan amount, Loan Sanction Amount (USD) (continuous).

Table 1: Dataset summary

Property	Value
Original rows (train.csv)	30,000
Total columns	21 (12 numeric, 9 categorical, incl. target)
Target column	Loan Sanction Amount (USD) (continuous)
Identifier columns dropped	Customer ID, Name, Property ID
Duplicate rows removed	14,950
Rows after de-duplication	15,050
Rows after outlier removal (IQR method)	13,520

Missing values. Several columns had substantial missingness before cleaning – most notably Type of Employment (7,270 missing), Income (USD) (4,576), Property Age (4,850), and Income Stability (1,683) – out of the original 30,000 rows. After the EDA pipeline’s imputation step (median for numeric columns, mode for categorical columns), all 21 columns had zero missing values.

3 Preprocessing Steps

The following preprocessing pipeline was applied:

1. **Identifier removal:** Customer ID, Name, and Property ID were dropped before modelling, as they carry no predictive signal.
2. **Missing value handling (train):** numeric columns were imputed with the training-set median and categorical columns with the training-set mode, reducing missingness from the counts above to zero across all columns.
3. **Missing value handling (test):** test.csv uses the placeholder string "?" (rather than a blank cell) for missing values in several columns. These were replaced with NaN, the affected columns were coerced back to numeric dtype, and the same train-fitted median/mode imputers were applied – without this step, SimpleImputer fails with "could not convert string to float: '?'" since the column is read in as text.
4. **Sentinel-value cleaning:** –999 values in Property Price and the target column were converted to NaN; rows with a missing target or missing Property Price were dropped from the training set.
5. **Duplicate removal:** 14,950 duplicate rows were removed, reducing the training set from 30,000 to 15,050 rows.
6. **Outlier removal (IQR method):** for every numeric feature, values outside $[Q1 - 1.5 \text{ IQR}, Q3 + 1.5 \text{ IQR}]$ were flagged and removed. No. of Defaults and Co-Applicant were skipped (IQR = 0, sparse/near-constant columns). This step reduced the training set from 15,050 to 13,520 rows (634, 367, 406, 268, 634, 434, and 281 outliers flagged in Income (USD), Loan Amount Request (USD), Current Loan Expenses (USD), Dependents, Property Age, Property Price, and the target, respectively).
7. **Categorical encoding:** all categorical columns were one-hot encoded (drop first=True); train and test were then column-aligned so both share the exact same one-hot feature set.
8. **Feature standardization:** all features were scaled with StandardScaler, fit once on the full training set and reused for the test set, since Ridge/Lasso/Elastic Net penalize coefficient magnitude directly and are therefore scale-sensitive.
9. **Train/validation split:** an additional 80/20 split of the training data was held out purely to plot the training-vs-validation error curve (Figure 1, panel e); all cross-validation and final-model results use the full 13,520-row training set.

4 Implementation Details

4.1 Models

- **Linear Regression** – ordinary least squares baseline, no regularization.
- **Ridge Regression** – ℓ_2 penalty $\alpha \sum_j \beta_j^2$.
- **Lasso Regression** – ℓ_1 penalty $\alpha \sum_j |\beta_j|$, capable of driving coefficients exactly to zero.
- **Elastic Net** – combined penalty $\alpha \left(r \sum_j |\beta_j| + \frac{1-r}{2} \sum_j \beta_j^2 \right)$.

4.2 Hyperparameter Search Space

Table 2: Hyperparameter search space

Model	Hyperparameter	Values
Ridge Regression	α	{0.01, 0.1, 1, 10, 100}
Lasso Regression	α	{0.001, 0.01, 0.1, 1, 10}
Elastic Net	α	{0.01, 0.1, 1, 10}
	ℓ_1 ratio	{0.2, 0.5, 0.8}

4.3 Search Method

Grid Search with 5-fold cross-validation (GridSearchCV, scoring = R^2 , run in parallel across all available CPU cores) was used for Ridge, Lasso, and Elastic Net. Linear Regression has no hyperparameters and was fit directly as the baseline.

5 Visualizations

All required visualizations were generated in a single combined dashboard figure (Figure 1), consisting of six panels:

- **(a) Target variable distribution.** The histogram (with KDE overlay) of Loan Sanction Amount (USD) is strongly right-skewed and, more importantly, clearly **bimodal**: a large spike of roughly 4,600+ loans sanctioned at or near \$0, followed by a much smaller, broader hump of loans sanctioned in the \$10,000–\$175,000 range. This means a substantial fraction of applications in the training data were effectively rejected (or sanctioned a negligible amount), while the remainder were sanctioned roughly proportionally to their request – two very different populations sharing one target column.
- **(b) Loan Amount Request vs. Loan Sanction Amount.** This confirms Loan Amount Request (USD) as the feature most correlated with the target (matching its dominant coefficient in Table 7). The scatter shows exactly the two populations from panel (a): a dense diagonal band where sanctioned amount scales almost linearly with requested amount (up to $\sim$ \$175,000 sanctioned for $\sim$ \$200,000+ requested), sitting above a second, equally dense band running flat along Sanctioned $\approx$ \$0 across the *entire* range of requested amounts – i.e. loans of every size that were granted essentially nothing.
- **(c) Predicted vs. Actual (Lasso Regression).** The model with the best CV R^2 (0.4596, Table 4) is plotted against the $y = x$ reference line. The two-population structure from (a)/(b) is directly visible here: a vertical cluster of points sits at Actual $\approx$ \$0 but is **predicted anywhere from about –\$20,000 to over \$100,000** – the model

cannot identify which "near-zero" applications belong to the rejected population. For the proportional population, points form a diagonal band that runs consistently *below* the $y = x$ line at higher amounts, i.e. the model systematically **underpredicts** large sanctioned amounts.

- **(d) Residual Plot (Lasso Regression).** This is the clearest evidence of model misspecification: rather than a random scatter around zero, residuals form two distinct diagonal bands – one trending from \$0 up to ~\$50,000 as predicted value increases, and a separate band trending sharply downward from \$0 to about -\$100,000. This "fan" shape is the residual-space signature of the same rejected-vs-proportional mixture: the downward band is the rejected population (actual ≈ 0 , so residual becomes more negative the more the model predicts), while the upward band is the underpredicted proportional population. A well-specified linear model would show residuals scattered randomly around zero with no visible structure; this is the opposite of that.
- **(e) Training vs. validation error.** Train MSE ($\approx 8.5 \times 10^8$) and validation MSE ($\approx 8.3 \times 10^8$, marginally *lower* than train) are nearly identical for all four models, with essentially no gap. This rules out overfitting entirely – consistent with Section 7 – and instead confirms all four models are hitting the same underfitting ceiling caused by the bimodal target structure in (a)/(b), not by fitting noise in the training data.
- **(f) Coefficient comparison (top 10 features).** Loan Amount Request (USD) ($\sim 22,000$) and Credit Score ($\sim 10,900$) dominate by more than an order of magnitude over every other feature shown – Type of Employment Managers, Property Price, Profession Working, Co-Applicant, Age, Current Loan Expenses (USD), Type of Employment Laborers, and Type of Employment_HR staff – all of which sit visually near zero on the same axis, matching Table 7 exactly.

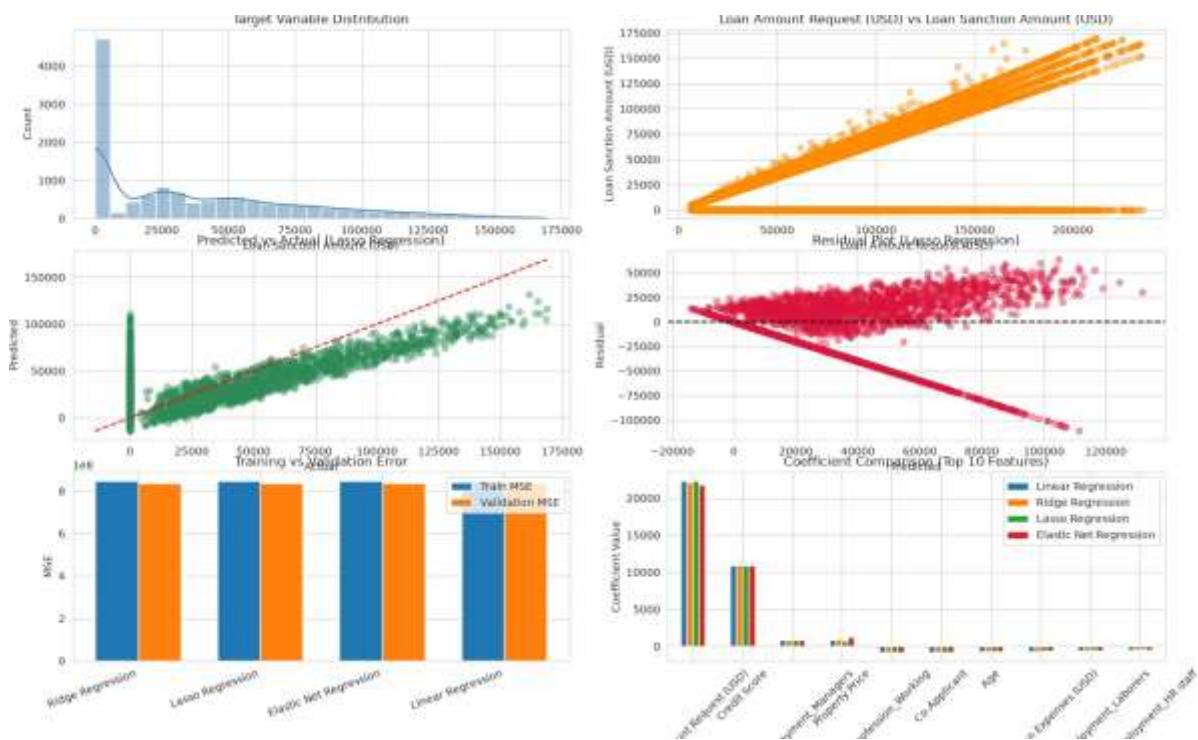


Figure 1: Combined regression dashboard: target distribution, feature-vs-target scatter, predicted-vs-actual and residual plots (best model, Lasso Regression), training-vs-validation error, and coefficient comparison.

6 Performance Tables

6.1 Hyperparameter Tuning Results

Table 3: Hyperparameter tuning summary

Model	Search Method	Best Parameters	Best CV R^2
Ridge Regression	Grid Search	$\alpha = 10$	0.4595
Lasso Regression	Grid Search	$\alpha = 10$	0.4596
Elastic Net Regression	Grid Search	$\alpha = 0.01$, ℓ_1 ratio = 0.8	0.4595

6.2 Cross-Validation Performance (K = 5)

Table 4: 5-fold cross-validation performance (full training set, $n = 13,520$)

Model	MAE	MSE	RMSE	R^2
Linear Regression	22,251.52	8.4946×10^8	29,145.56	0.4595
Ridge Regression	22,253.66	8.4946×10^8	29,145.48	0.4595
Lasso Regression	22,250.21	8.4931×10^8	29,142.92	0.4596
Elastic Net Regression	22,256.10	8.4947×10^8	29,145.60	0.4595

6.3 Test Set Performance

Table 5: Held-out test set performance

Model	MAE	MSE	RMSE	R^2	Training Time (s)
<i>Not available – test.csv has no target column (Kaggle competition test set).</i>					

test.csv is an unlabeled competition test set. Predictions were generated by all four fitted models for every test row, but ground-truth metrics cannot be computed without the true sanctioned amounts.

6.4 Training Time

Table 6: Training time (final models, fit on the 80% train split)

Model	Training Time (s)
Ridge Regression	0.0060
Linear Regression	0.1084
Lasso Regression	0.4975
Elastic Net Regression	2.1238

Ridge is essentially instantaneous (closed-form solution); Lasso and Elastic Net are slower since both rely on iterative coordinate descent to handle the non-differentiable ℓ_1 penalty, with Elastic Net the slowest as it must balance both penalty terms simultaneously.

6.5 Effect of Regularization on Coefficients

Table 7: Coefficient comparison across all four models (standardized features)

Feature	Linear	Ridge	Lasso	Elastic Net
Age	−683.85	−683.53	−674.55	−682.10
Income (USD)	−225.55	−224.35	−21.63	−222.27
Loan Amount Request (USD)	22,358.89	22,144.93	22,336.28	21,806.00
Current Loan Expenses (USD)	−661.75	−634.96	−626.84	−591.72
Dependents	−351.08	−350.05	−340.49	−348.39
Credit Score	10,940.37	10,935.23	10,941.57	10,926.55
No. of Defaults	−312.64	−313.05	−302.76	−313.66
Property Age	−225.55	−224.35	−415.98	−222.53
Property Type	−200.48	−200.51	−190.80	−200.54
Co-Applicant	−755.22	−754.92	−745.45	−754.41
Property Price	811.24	996.24	795.61	1,288.10
Gender_M	−65.78	−65.04	−55.47	−63.87
Profession_State servant	−294.35	−293.13	−270.11	−291.12
Profession_Working	−785.62	−784.52	−767.79	−782.72
Type of Employment_Cleaning staff	−265.71	−265.73	−234.59	−265.66
Type of Employment_Cooking staff	−113.33	−114.45	−78.11	−116.17
Type of Employment_Core staff	−37.73	−37.75	3.10	−37.63
Type of Employment_Drivers	−340.57	−342.89	−297.19	−346.56
Type of Employment_HR staff	−429.87	−429.51	−411.63	−428.89
Type of Employment_High skill tech	−171.81	−171.84	−127.42	−171.75
Type of Employment_IT staff	130.64	130.80	127.23	131.06
Type of Employment_Laborers	−596.74	−598.64	−524.00	−601.56
Type of Employment_Low-skill Laborers	12.10	11.33	14.54	10.08
Type of Employment_Managers	817.63	816.35	847.40	814.40
Type of Employment_Medicine staff	101.77	101.06	120.05	100.02
Type of Employment_Private service	−354.08	−354.32	−325.20	−354.64
Type of Employment_Realty agents	−88.81	−88.57	−68.92	−88.16
Type of Employment_Sales staff	−140.52	−142.52	−71.81	−145.58
Type of Employment_Secretaries	182.94	182.82	184.59	182.64
Type of Employment_Security staff	−159.02	−158.80	−124.44	−158.38
Type of Employment_Waiters/barmen staff	385.91	384.79	388.89	382.97
Location_Semi-Urban	33.19	33.33	27.74	33.60
Location_Urban	−306.71	−305.22	−300.46	−302.78
Expense Type 1_Y	−346.90	−348.06	−340.15	−349.89
Expense Type 2_Y	393.84	392.94	381.43	391.49
Has Active Credit Card_Inactive	356.48	356.75	337.21	357.15
Has Active Credit Card_Unpossessed	317.35	317.03	298.62	316.47
Property Location_Semi-Urban	−146.71	−146.11	−139.40	−145.13
Property Location_Urban	109.58	110.08	103.34	110.89

7 Overfitting and Underfitting Analysis

Cross-validation performance is essentially flat across all four models. Linear, Ridge, Lasso, and Elastic Net all land within $0.0001 R^2$ of each other (0.4595–0.4596, Table 4), and their coefficient vectors (Table 7) are nearly identical in both sign and magnitude. This is a

strong signal that regularization is not the limiting factor here: adding an ℓ_2 , ℓ_1 , or combined penalty barely moves the model away from the unregularized OLS solution.

Selected regularization strengths support this. Grid search selected $\alpha = 10$ for both Ridge and Lasso (mid-range of their search grids) and a very small $\alpha = 0.01$ for Elastic Net, meaning only light shrinkage was preferred in every case. If overfitting were the main problem, we would expect stronger regularization ($\alpha = 100$ for Ridge, or Lasso zeroing out many coefficients) to noticeably outperform the baseline – it doesn't. The only feature Lasso drove meaningfully toward zero was Income (USD) (from -225.5 down to -21.6) and Type of Employment Core staff (from -37.7 to $+3.1$, i.e. effectively removed); every other coefficient survives largely intact.

This pattern points to underfitting, not overfitting – and the dashboard shows exactly why. An R^2 of ≈ 0.46 means the entire linear family explains under half of the variance in sanctioned loan amount, and four different regularization strategies converge to essentially the same, mediocre fit. Panel (a) of Figure 1 reveals the target variable is **bimodal**: a large cluster of loans sanctioned at or near \$0 sits alongside a separate population sanctioned roughly proportionally to the requested amount. Panel (d)'s residual plot shows the direct consequence – residuals split into two clean diagonal bands rather than scattering randomly around zero, which is the textbook signature of a single linear model trying to fit two structurally different subpopulations at once. Panel (e) confirms train and validation MSE are essentially equal for every model, so this is not overfitting to noise; it is the model architecture itself (one straight-line relationship) being unable to represent a target that behaves like a mixture of "rejected" and "approved-proportionally" cases. Since a held-out test-set R^2 could not be computed (Table 5), a direct CV-test gap cannot be measured here, but the flat CV score combined with the structured residuals is stronger evidence of underfitting than a train-test gap would have been.

8 Bias–Variance Analysis

High bias, low variance across the board. All four models converge to nearly the same coefficients and the same CV R^2 regardless of regularization strength. Low variance between models built with very different α values – including $\alpha = 0$ (Linear Regression) all the way to $\alpha = 10$ (Ridge/Lasso) – indicates the training data itself is not noisy enough, relative to the model's linear form, for regularization to meaningfully change the fitted solution. In other words, the models are not struggling with variance; they are all hitting the same bias ceiling imposed by assuming a linear relationship between the (standardized, one-hot encoded) features and loan sanction amount.

Ridge (variance control, minimal effect here). Ridge's smooth ℓ_2 shrinkage barely perturbs the OLS coefficients (e.g. Loan Amount Request (USD): $22,358.89 \rightarrow 22,144.93$), confirming there was little variance for it to reduce.

Lasso (feature sparsity, minimal effect here). Of 38 one-hot encoded features, Lasso only meaningfully shrank two toward zero (Income (USD) and Type of Employment Core staff); it did not perform aggressive feature selection, and marginally improved CV R^2 (0.4596 vs. 0.4595) over the other three – the smallest measurable edge, consistent with its slightly better handling of a couple of weakly informative features rather than any meaningful bias-variance trade-off.

Elastic Net (blend, minimal effect here). With ℓ_1 ratio = 0.8, Elastic Net leans mostly Lasso-like but at a much smaller $\alpha = 0.01$, and its coefficients and R^2 sit essentially on top of the other three models – as expected for a blend, but here the blend adds no measurable benefit since neither penalty was doing much work to begin with.

Conclusion of this analysis. The dominant issue in this experiment is **bias, not variance**: two features (Loan Amount Request (USD) and Credit Score) carry coefficients an

order of magnitude larger than everything else, and the model still only reaches $R^2 \approx 0.46$. Panels (a), (c), and (d) of the dashboard identify the specific source of that bias: the target variable is a mixture of two populations (loans sanctioned near \$0 vs. loans sanctioned roughly proportionally to the request), and a single linear equation cannot represent both regimes at once – it compromises between them, systematically underpredicting the proportional group and overpredicting the near-zero group. No amount of ℓ_1/ℓ_2 regularization on this feature set will close that gap, since the problem is the functional form, not coefficient magnitude.

9 Observations and Conclusion

- **Best-performing model (by CV R^2): Lasso Regression** ($\alpha = 10$) marginally leads with $R^2 = 0.4596$ and the lowest MAE/RMSE among the four models, though the margin over Ridge, Elastic Net, and even unregularized Linear Regression is negligible ($< 0.0002 R^2$).
- **Regularization had minimal impact:** all four models converge to nearly identical coefficients and CV scores. The chosen hyperparameters ($\alpha = 10$ for Ridge/Lasso, $\alpha = 0.01$ for Elastic Net) indicate the search found no benefit from strong shrinkage – confirming the model is not overfitting the (already outlier-filtered) 13,520-row training set.
- **Dominant predictors:** Loan Amount Request (USD) ($\sim 22,000$ – $22,400$) and Credit Score ($\sim 10,900$ – $10,950$) dwarf every other coefficient by more than an order of magnitude, making them by far the strongest linear predictors of sanctioned loan amount.
- **Bias–variance trade-off :** the near-identical performance across regularization strengths points to a **high-bias, low-variance** regime – the linear model family itself, not overfitting, is capping performance around $R^2 \approx 0.46$.
- **Training time:** Ridge trains near-instantly (0.006s, closed-form solution); Lasso and Elastic Net are slower (0.50s, 2.12s) due to iterative coordinate descent, but all four remain fast enough that training cost is not a practical concern at this dataset size.
- **Overall recommendation:** given the negligible difference between models, **Ridge Regression** is the most practical choice – it trains fastest, its performance is statistically indistinguishable from Lasso/Elastic Net, and unlike Lasso it does not risk zeroing out a feature that could matter on unseen data.
- **Limitation and future work:** an R^2 around 0.46 leaves more than half the variance in loan sanction amount unexplained, and Figure 1 shows why – the target is effectively a mixture of “rejected” and “approved-proportionally” loans, which no single linear equation can represent well. Natural next steps include (i) a **two-stage model**: first classify whether a loan is sanctioned at all (logistic regression / classifier), then regress the amount only for the approved subset; (ii) engineering interaction features (e.g. income-to-loan-request ratio); (iii) trying a non-linear model (e.g. gradient boosting or random forest) that can natively split on this kind of regime change; and (iv) obtaining true labels for test.csv, or carving out a labeled held-out split from train.csv, so that genuine test-set generalization (Table 5) can finally be measured.